Cloover Full-Stack Coding Challenge

Project: GreenQuote - solar financing pre-qualification

Aim for ~4 hours for the core implementation. You may use AI coding assistants. In the next round you'll be asked to explain your code line-by-line and extend it live without AI.

Goal

Build a minimal end-to-end app that lets an authenticated user request a **pre-qualification quote** for a residential solar system and view results. The solution should cover: UI, API, DB, auth, basic infra/ops, and a small test suite.

Suggested stack (TypeScript encouraged):

- **Frontend:** Next.js (recommended) or any modern React/Vue setup.
- **Backend:** Next.js API routes, Node/Express, or NestJS.
- **DB:** SQLite or Postgres (Docker).
- **Auth:** Email/password as the baseline (required). Optional OAuth (see below).

Keep it simple and conventional; choose tools you can set up fast.

What to build

A) Frontend

1. **Quote form**

   Fields (with client-side validation):

   - fullName, email (pre-filled based on the logged-in user)
   - address
   - monthlyConsumptionKwh (number)
   - systemSizeKw (number)

- downPayment (number, optional)

  Action: **Get pre-qualification** (calls your API).

2. **Quote results**

   Display:

   - Computed **system price**
   - **Risk band** (A/B/C)
   - Three installment **offers** (terms **5/10/15** years) with **monthly payment** values (see “Pricing model” below).

3. **Quotes list & Admin view (protected)**

   - **My quotes** (/quotes): after sign-in, show a table of **only the current user’s** quotes (date, system size, price, band, link to details).
   - **Admin view** (/admin/quotes): admin users can view **all quotes** and **filter by user** (search by name/email or a dropdown).
   - Authorization must be enforced **server-side** (not just hidden UI). Seed one admin in your DB (e.g., admin@test.com).

Accessibility and clear presentation matter. Keep the UI simple and responsive.

B) Backend API

Implement at least these endpoints:

- **POST /api/quotes**
  - Validates input.
  - Applies the **pricing model** (below).
  - Persists the request + computed result.
  - Returns a normalized JSON response (inputs, derived values, offers).
- **GET /api/quotes/:id** – fetch one (auth required; enforce ownership unless admin).
- **GET /api/health** – liveness check.

Pricing model

- **System price:** systemPrice = systemSizeKw * 1200 (choose any currency and stick to it).
- **Principal amount:** systemPrice minus downPayment.
- **Risk band (coarse):**
  - **A** if monthlyConsumptionKwh ≥ 400 and systemSizeKw ≤ 6
  - **B** otherwise if monthlyConsumptionKwh ≥ 250
  - **C** otherwise
- **Base APR by band:** A → **6.9%**, B → **8.9%**, C → **11.9%**
- **Offers:** terms **5**, **10**, **15** years. For each term, compute the monthly payment using the **standard amortization calculation**.

Return each offer with: termYears, apr, principalUsed, and monthlyPayment.

C) Authentication & Authorization

Required baseline (email/password):

- **Registration:** minimal form with **fullName, email, password**.
- **Login:** email + password.
- **Session:** keep required details in the session (cookie-based session or JWT).
- Protect routes/endpoints that create or list quotes.

Bonus: Integrate an IdP such as **Keycloak**.

D) Database & schema

- Minimal tables: User and Quote
- Include a migration/schema and any seed data (including one admin user).

E) Infra & operations

- **Local dev:** Provide a **Dockerfile** (and optionally docker-compose.yml) or clearly documented scripts to run API + DB locally.

- **Config:** 12-factor envs; include a **.env.example**.
- **Observability:**
  - structured **logging** around request/response/error paths;
  - implement **/api/health**.

F) Quality gates

- **Automated tests:** Reasonable unit and integration tests
- **Lint/format:** set up what you normally use.
- **Docs:** a concise **README** describing setup/run/test steps, trade-offs, and "what you'd do next."

Deliverables

- Repo (GitHub link or zip) including:
  - Source code and **Git history**.
  - **README** with prerequisites, run/test instructions, and a brief design/decisions note.
  - **.env.example**, schema/migrations, and any seed scripts.
  - Short **API reference** (markdown or OpenAPI/Swagger-either is fine).

Bonus

Attempt only after the core implementation is strong.

Pick one or two-optional extras that showcase breadth:

- **OpenAPI** auto-rendered docs page.
- **E2E test** (e.g., Playwright) covering sign-in → quote → view results.
- Implement an **amortization schedule** which the user can view for the selected quote.
- **PDF Export:** Allow users to download their quote (including offers and optionally amortization schedule) as a formatted PDF.

How we review

We'll assess along five themes:

1. **Correctness** - adheres to the core requirements.
2. **Documentation** - clear README and commit history over excessive code comments.
3. **Testing** - reasonable mix of unit/integration tests.
4. **Readability** - naming, structure, duplication, consistency.
5. **Application architecture** - conventional patterns for your stack; sensible modularization and boundaries.

Questions we may ask

- Why this stack and architecture? What trade-offs did you make?
- If you had **another day**, what would you add or change first?
- How would you implement **CI/CD** and what would you test in the pipeline?
- Where would you **deploy on GCP** and why (Cloud Run, GKE, App Engine)?
- How did you approach testing, and what would you add for production readiness?